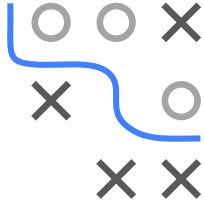
Introduction to Deep Learning



Recurrent Neural Networks
Attention and Transformers



A woman is throwing a frisbee in a park.

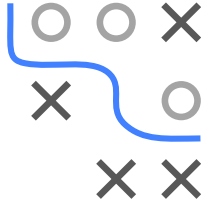


A stop sign is on a road with a mountain in the background.

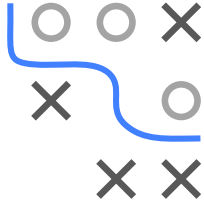
Learning goals

- Familiarize with the most recent sequence data modeling technique:
  - Attention Mechanism
  - Transformers
- Get to know the CNN alternative to RNNs

Attention

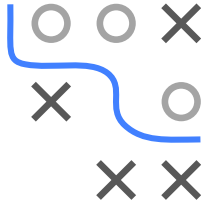


ATTENTION I



- In a classical decoder-encoder RNN all information about the input sequence must be incorporated into the final hidden state, which is then passed as an input to the decoder network.
- With a long input sequence this fixed-sized context vector is unlikely to capture all relevant information about the past.
- Each hidden state contains mostly information from recent inputs.
- Key idea: Allow the decoder to access all the hidden states of the encoder (instead of just the final one) so that it can dynamically decide which ones are relevant at each time-step in the decoding.
- This means the decoder can choose to "focus" on different hidden states (of the encoder) at different time-steps of the decoding process similar to how the human eye can focus on different regions of the visual field.
- This is known as an **attention mechanism**.

ATTENTION II



- The attention mechanism is implemented by an additional component in the decoder.
- For example, this can be a simple single-hidden layer feed-forward neural network which is trained along with the RNN.
- At any given time-step *i* of the decoding process, the network computes the relevance of encoder state $\mathbf{z}^{[j]}$ as:

$$rel(\mathbf{z}^{[j]})^{[i]} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\mathbf{g}^{[i-1]}; \mathbf{z}^{[j]}])$$

  where $\mathbf{v}_a$ and $\mathbf{W}_a$ are the parameters of the feed-forward network, $\mathbf{g}^{[i-1]}$ is the decoder state from the previous time-step and ';' indicates concatenation.
- The relevance scores (for all the encoder hidden states) are then normalized which gives the *attention weights* $(\alpha^{[j]})^{[i]}$:

$$(\alpha^{[j]})^{[i]} = \frac{\exp(rel(\mathbf{z}^{[j]})^{[i]})}{\sum_{j'} \exp(rel(\mathbf{z}^{[j']})^{[i]})}$$

ATTENTION III

- The attention mechanism allows the decoder network to focus on different parts of the input sequence by adding connections from all hidden states of the encoder to each hidden state of the decoder.

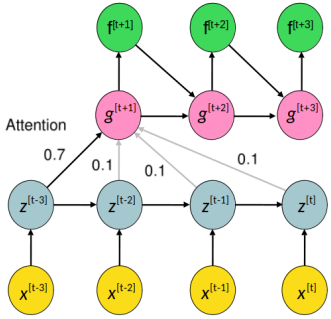


Figure: Attention at $i = t + 1$

ATTENTION IV

- At each time step i, a set of weights $(\alpha^{[j]})^{[i]}$ is computed which determine how to combine the hidden states of the encoder into a context vector $\mathbf{g}^{[i]} = \sum_{j=1}^{n_x} (\alpha^{[j]})^{[i]} \mathbf{z}^{[j]}$, which holds the necessary information to predict the correct output.

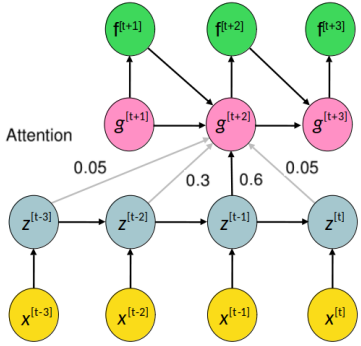
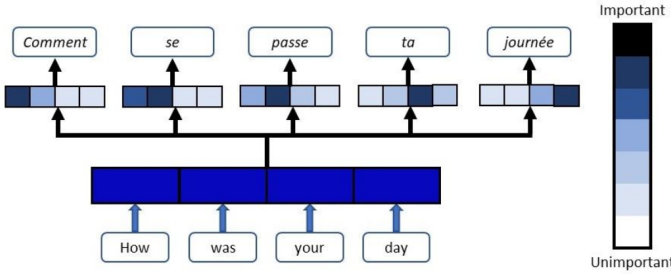


Figure: Attention at $i = t + 2$

ATTENTION V



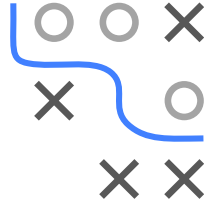


Figure: An illustration of a machine translation task using an encoder-decoder model with an attention mechanism. The attention weights at each time-step of the decoding/translation process indicate which parts of the input sequence are most relevant. There are 4 attention weights because there are 4 encoder states (Loye, 2019).

ATTENTION



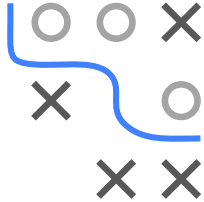
A woman is throwing a frisbee in a park.



A stop sign is on a road with a mountain in the background.

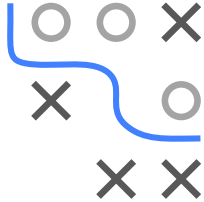
Figure: Attention for image captioning: the attention mechanism tells the network roughly which pixels to pay attention to when writing the text (Xu et al., 2016).

Transformers



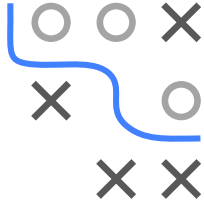
TRANSFORMERS I

- Advanced RNNs have similar limitations as vanilla RNN networks:
  - RNNs process the input data sequentially.
  - Difficulties in learning long term dependency (although GRU or LSTM perform better than vanilla RNNs, they sometimes struggle to remember the context introduced earlier in long sequences).
- These challenges are tackled by transformer networks.



TRANSFORMERS II

- Transformers are solely based on attention (no RNN or CNN).
- In fact, the paper which coined the term *transformer* is called *Attention is all you need*.
- They are the state-of-the-art networks in natural language processing (NLP) tasks since 2017.
- Transformer architectures like BERT (Bidirectional Encoder Representations from Transformers, 2018) and GPT-3 (Generative Pre-trained Transformer-3, 2020) are pre-trained on a large corpus and can be fine-tuned to specific language tasks.



TRANSFORMERS III

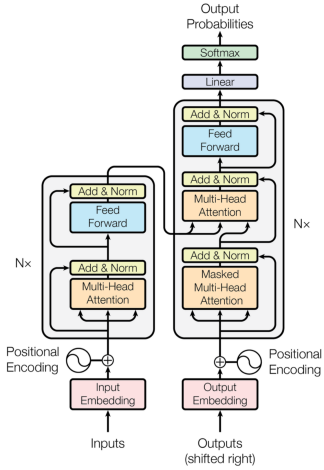


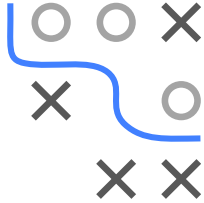
Figure 1: The Transformer - model architecture.

Figure: Example of a transformer architecture (Vaswani et al., 2023)

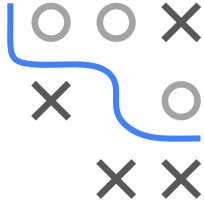
TRANSFORMER COMPONENTS I

In- and Output Embeddings:

- Each input (e.g., word or characters; referred to as token) in the input sequence is converted into a dense vector of fixed size.
- Embeddings capture semantic information about the tokens.
- In Transformers, input embeddings are combined with positional encodings to form the final input representations.
- Output Embeddings: In the decoder, predicted tokens are also converted into embeddings.



TRANSFORMER COMPONENTS II



Self-Attention:

- Different from (Cross-)Attention: Captures dependencies within a single sequence (cross-attention involves attending to a different sequence).
- Mathematically, for input X, self-attention is calculated as:

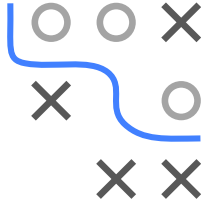
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  where Q, K, V are the queries, keys, and values derived from X.
- For a given input sequence of embeddings X:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

  with learned weights W_Q, W_K, W_V that project the embeddings into different subspaces.

TRANSFORMER COMPONENTS III



Positional Encoding:

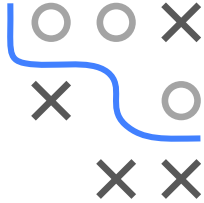
- Used to incorporate information about the position of tokens since Transformers process the entire sequence simultaneously.
- Computed using sine and cosine functions of different frequencies:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right),$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

  with *pos* being the position of the token, *d* the embedding dimensionality and *i* the dimension index.
- These encodings are added to the embeddings element-wise.

TRANSFORMER COMPONENTS IV



Masked Multi-Head Attention:

- Masking in the decoder: prevents positions from attending to subsequent positions, thereby leaking future values.
- Multi-head attention allows the model to jointly attend to information from different representation subspaces.
- For each head h:

$$\text{head}_h = \text{Attention}(Q^h, K^h, V^h)$$

- The outputs of all heads are concatenated and linearly transformed.

REFERENCES I

- Roy, R. (2019, February 4). *Temporal Convolutional Networks*. Medium. https://medium.com/@raushan2807/temporal-convolutional-networks-bfea16e6d7d2
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention Is All You Need.*
- Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., & Bengio, Y. (2016). *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention.*
- Loye, G. (2019, September 15). *Attention mechanism*. FloydHub Blog. https://blog.floydhub.com/attention-mechanism/

